

DESENVOLVIMENTO DE SISTEMAS WEB I

Aula 8: Conectando o ContatoDAO ao PostgreSQL via JDBC

Da lista em memória para dados de verdade — sem mudar quem usa o DAO

Aula prática: Connection Factory + CRUD completo em SQL

Objetivos da aula

1

Adicionar o driver JDBC do PostgreSQL ao projeto

Via dependência Maven.

2

Entender e construir uma Connection Factory

Um ponto único responsável por abrir conexões com o banco.

3

Reescrever todo o CRUD do ContatoDAO com JDBC

listar, buscarPorId, inserir, atualizar e excluir — agora em SQL de verdade.

4

Aplicar try-with-resources

Garantindo que conexões sejam sempre fechadas, mesmo em caso de erro.

Agenda da aula

1 Recap: o banco existe, falta conectar

2 Adicionar o driver JDBC no pom.xml

3 Por que uma Connection Factory

4 Construindo a ConnectionFactory

5 Reescrevendo listar() e buscarPorId()

6 Reescrevendo inserir(), atualizar() e excluir()

7 Boas práticas: try-with-resources

8 Testando o CRUD completo no banco real

Onde paramos

Banco criado

O banco agenda_telefonica e a tabela contato já existem no PostgreSQL (Aula 7).

Tabela vazia

A estrutura está pronta (id, nome, telefone, email) mas sem nenhum registro ainda.

DAO ainda em memória

O ContatoDAO continua usando a List<Contato> das Aulas 5 e 6.

A promessa do padrão DAO

Só o "de dentro" do DAO vai mudar — Servlets e JSPs não vão perceber nada (Aula 6).

1

Adicionar o driver JDBC do PostgreSQL

O driver é a biblioteca que sabe traduzir chamadas JDBC para o protocolo do PostgreSQL (Aula 6). Adicione no pom.xml:

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <version>42.7.13</version>
</dependency>
```

Depois de salvar o pom.xml, rode "Maven: Reload Project" no VSCode (ou clique no ícone de reload da aba Maven) para baixar a dependência.

Por que uma Connection Factory

Toda operação do DAO vai precisar de uma conexão com o banco. Em vez de repetir URL, usuário e senha em cinco métodos diferentes, centralizamos essa responsabilidade em uma única classe — uma "fábrica de conexões".

Um único lugar para configurar

Se a senha ou o endereço do banco mudar, é um único arquivo a atualizar.

Reaproveitável por qualquer DAO

Se o projeto crescer com outras tabelas, todos os DAOs usam a mesma fábrica.

Padrão de projeto conhecido como Factory

Uma classe cuja única responsabilidade é criar objetos — aqui, objetos Connection.

2

Construindo a ConnectionFactory

src/main/java/br/edu/faculdade/dao/ConnectionFactory.java

```
public class ConnectionFactory {

    private static final String URL =
        "jdbc:postgresql://localhost:5432/agenda_telefonica";
    private static final String USER = "postgres";
    private static final String PASSWORD = "sua_senha_aqui";

    public static Connection getConnection() {
        try {
            return DriverManager.getConnection(URL, USER, PASSWORD);
        } catch (SQLException e) {
            throw new RuntimeException("Erro ao conectar ao banco de dados", e);
        }
    }
}
```

A URL segue o formato `jdbc:postgresql://host:porta/nome_do_banco`. Em projetos reais, a senha nunca fica direto no código — mas para aprender o JDBC hoje, vamos manter simples.

3

Reescrevendo listar()

```
public static List<Contato> listar() {
    List<Contato> lista = new ArrayList<>();
    String sql = "SELECT * FROM contato ORDER BY id";

    try (Connection conn = ConnectionFactory.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql);
        ResultSet rs = stmt.executeQuery()) {

        while (rs.next()) {
            lista.add(new Contato(
                rs.getInt("id"),
                rs.getString("nome"),
                rs.getString("telefone"),
                rs.getString("email")));
        }

    } catch (SQLException e) {
        throw new RuntimeException("Erro ao listar contatos", e);
    }

    return lista;
}
```


4

Reescrevendo buscarPorId()

```
public static Contato buscarPorId(int id) {  
    String sql = "SELECT * FROM contato WHERE id = ?";  
  
    try (Connection conn = ConnectionFactory.getConnection();  
        PreparedStatement stmt = conn.prepareStatement(sql)) {  
        stmt.setInt(1, id);  
  
        try (ResultSet rs = stmt.executeQuery()) {  
            if (rs.next()) {  
                return new Contato(rs.getInt("id"), rs.getString("nome"),  
                                    rs.getString("telefone"), rs.getString("email"));  
            }  
        }  
    } catch (SQLException e) {  
        throw new RuntimeException("Erro ao buscar contato", e);  
    }  
    return null;  
}
```

O "?" no SQL é um parâmetro do PreparedStatement — stmt.setInt(1, id) preenche esse espaço de forma segura, protegendo contra SQL Injection (mencionado na Aula 6).

5 Reescrevendo inserir()

```
public static void inserir(Contato contato) {  
    String sql = "INSERT INTO contato (nome, telefone, email) VALUES (?, ?, ?)";  
  
    try (Connection conn = ConnectionFactory.getConnection();  
        PreparedStatement stmt = conn.prepareStatement(sql)) {  
        stmt.setString(1, contato.getNome());  
        stmt.setString(2, contato.getTelefone());  
        stmt.setString(3, contato.getEmail());  
        stmt.executeUpdate();  
    } catch (SQLException e) {  
        throw new RuntimeException("Erro ao inserir contato", e);  
    }  
}
```

Reparem: não passamos o id! A coluna é serial (Aula 7) — o próprio PostgreSQL numera sozinho. É exatamente a substituição do proximoId manual do ContatoDAO em memória (Aula 6).

6

Reescrevendo atualizar() e excluir()

```
public static void atualizar(Contato atualizado) {
    String sql = "UPDATE contato SET nome=?, telefone=?, email=? WHERE id=?";
    try (Connection conn = ConnectionFactory.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setString(1, atualizado.getNome());
        stmt.setString(2, atualizado.getTelefone());
        stmt.setString(3, atualizado.getEmail());
        stmt.setInt(4, atualizado.getId());
        stmt.executeUpdate();
    } catch (SQLException e) {
        throw new RuntimeException("Erro ao atualizar contato", e);
    }
}
```

```
public static void excluir(int id) {
    String sql = "DELETE FROM contato WHERE id = ?";
    try (Connection conn = ConnectionFactory.getConnection();
        PreparedStatement stmt = conn.prepareStatement(sql)) {
        stmt.setInt(1, id);
        stmt.executeUpdate();
    } catch (SQLException e) {
        throw new RuntimeException("Erro ao excluir contato", e);
    }
}
```

Por que try (...) e não try/finally

Connection, PreparedStatement e ResultSet mantêm recursos abertos (memória, conexão de rede). Esquecer de fechá-los é uma das causas mais comuns de aplicações Java travarem em produção.

✗ Jeito antigo (verboso)

```
Connection conn = null;
try {
    conn = ...;
    // usar conn
} finally {
    if (conn != null) {
        conn.close();
    }
}
```

✓ try-with-resources

```
try (Connection conn = ...) {
    // usar conn
}
// fecha sozinho aqui,
// mesmo se der exceção
```

Testando o CRUD completo no banco real

- 1 Suba o Tomcat e acesse a Agenda pelo navegador.
- 2 Clique em "Novo Contato" e salve um contato — ele deve aparecer na listagem.
- 3 Edite esse contato e confirme que os dados mudaram.
- 4 Exclua o contato e confirme que ele some da lista.
- 5 Reinicie o Tomcat: os dados continuam lá. Esse é o ganho de hoje!

```
-- Conferindo direto no banco:  
SELECT * FROM contato;
```

MARCO ATINGIDO

A Agenda agora tem memória de verdade.

- 1 Hoje: ConnectionFactory + CRUD inteiro do ContatoDAO com JDBC e PostgreSQL
- 2 Os dados agora sobrevivem a reinícios — persistência real, pela primeira vez
- 3 Próxima aula: criação de endpoints REST, para a Agenda também falar JSON